TECHNICAL SPECIFICATIONS & ARCHITECTURAL BLUEPRINT

Diwas Dinesh Rathod Personal Portfolio Engine

Document Purpose: This document provides an exhaustive, line-by-line technical specification and architectural teardown of the high-performance personal portfolio website engineered for Diwas Dinesh Rathod. It delineates the underlying software engineering paradigms, responsive system mechanisms, style systems, high-frequency scroll interception, and custom WebGL/Canvas preloading and rendering engines that compose this state-of-the-art web application.

DEVELOPER & ARCHITECT

Diwas Dinesh Rathod
Email: diwasrathour@gmail.com

FRAMEWORK PLATFORM

Next.js 16.2.6 // React 19.2.4
React Server Components (RSC)

STYLE PIPELINE

Tailwind CSS v4 (PostCSS Engine)
Glassmorphism & Fluid Typography

INTERACTION LAYERS

Framer Motion 12.4.0 (Custom Spring)
2D DPI-Aware Canvas Scrollytelling

DOCUMENT CLASSIFICATION

LAST ARCHITECTURAL STAMP

Engineering Blueprint // RESTRICTED *May 27, 2026 // Production Release*

1. Executive Tech Stack & Architecture

The web platform designed for Diwas Dinesh Rathod represents a modern, performance-first portfolio crafted on top of **Next.js 16.2.6** and **React 19.2.4**. It utilizes the Next.js App Router paradigm, optimizing the separation of static content and dynamic client interfaces through React Server Components (RSC) combined with granular client-side interaction points. Rather than relying on heavy third-party runtimes or complex heavy web libraries, the application is strictly optimized for fast loading and extremely smooth animation profiles.

Component / Layer	Technology Selected	Technical Purpose & Benefit
Core Architecture	Next.js 16.2.6 (App Router)	Enables zero-bundle-size static page generation (SSG), file-system based routing, and rapid hydration.
UI & Framework State	React 19.2.4	Optimized lightweight virtual DOM management with direct concurrent rendering pipelines.
Styling Engine	Tailwind CSS v4	Utilizes a completely redesigned compile-time engine, native CSS parser, CSS-first config imports, and minimal build size.
Animation Runtime	Framer Motion 12.4.0	Provides a declarative syntax for canvas transitions, spring damping curves, and GPU-accelerated motion values.
Icon Vector Assets	Lucide React 1.16.0	Clean SVG icons built as lightweight react components supporting treeshaking and CSS overrides.
Visual Canvas Pipeline	HTML5 Canvas 2D + Custom Hook	Bypasses standard React state tree updates to render a preloaded high-fidelity 74-frame cinematic sequence at high FPS.

Key Architectural Philosophies:

- **Minimal Hydration Overhead:** Interactive sub-trees are strictly labeled with the `'use client'` directive, keeping the initial JavaScript bundle exceptionally lean.
- **High-DPI Performance:** Painting is directed through a raw Canvas 2D Context, bypassing the React component tree and re-rendering operations entirely during scrolling.
- **CSS-First Styling:** Moving from Tailwind v3 JS-centric configuration to Tailwind v4 CSS-first declaration avoids configuration overhead and speeds up runtime rendering.

2. Cinematic Canvas Scrollytelling Pipeline

The centerpiece of the landing page experience is a **500vh Cinematic Canvas Scrollytelling Section**. As the user scroll/swipe triggers inputs, they do not scroll the webpage in a standard fashion. Instead, scrolling is intercepted, locking the scroll viewport in place while scrubbing through a 74-frame high-resolution image sequence. This visual story outlines the professional journey of Diwas Dinesh Rathod.

A. Preloading & Image Cache Pipeline

To guarantee stutter-free playback, a custom preloader hooks into the React lifecycle during initial mount. A sequence of 74 distinct, compressed, high-fidelity WebP images (`frame_00.webp` to `frame_73.webp`) is fetched asynchronously. A React progress bar tracks the loading percentage in real time. Once all promises in the preloading queue are successfully resolved, the image references are written directly to a mutable reference object (`imagesRef.current`) to prevent React component re-renders from clearing the preloaded cache.

B. Spring-Interpolated Deceleration Mechanics

To achieve a cinematic, buttery-smooth flow, the raw scroll values are decoupled from the canvas painter. When scroll actions occur, they modify a Framer Motion **MotionValue** called `sequenceProgress` (0.0 to 1.0). A secondary motion value, `smoothProgress`, is computed using the `useSpring` hook, configured with a stiffness of **60** and a damping of **22**. This spring setup acts as a high-frequency low-pass filter, smoothing out erratic mousewheel clicks or touch actions and decelerating into each frame with beautiful ease-out physics.

C. High-DPI Aware Canvas 2D Painting

During rendering, the screen resolution is dynamically queried via `window.devicePixelRatio` (typically 2x for Apple Retina screens). The HTML5 Canvas's internal backing store width and height are multiplied by this ratio, while its style properties match the display dimensions. The 2D rendering context is then scaled globally using `ctx.scale(dpr, dpr)`. This ensures that text overlay boundaries, graphic edges, and the sequence frames remain tack-sharp without any blurry bilinear interpolation artifacts.

D. Object-Fit 'Cover' Math in Canvas 2D Context

To keep the sequence background responsive under arbitrary browser window aspects (ultrawide, desktop, portrait mobile), an algorithmic equivalent of CSS's `object-fit: cover` is executed on every frame draw. The formula is:

```
// Aspect Ratio Math implemented in ScrollyCanvas.tsx const imgRatio = imgWidth / imgHeight; const canvasRatio = canvasWidth / canvasHeight; let drawWidth = canvasWidth; let drawHeight = canvasHeight; let offsetX = 0; let offsetY = 0; if (imgRatio > canvasRatio) { drawWidth = canvasHeight * imgRatio; offsetX = (canvasWidth - drawWidth) / 2; } else { drawHeight = canvasWidth / imgRatio; offsetY = (canvasHeight - drawHeight) / 2; } ctx.clearRect(0, 0, canvasWidth, canvasHeight); ctx.drawImage(img, offsetX, offsetY, drawWidth, drawHeight);
```

3. Advanced Scroll Locking & Event Hijacking

Standard CSS scroll-snap structures do not allow for custom spring-damped interactive scrubbing. To solve this, the page scroll container acts as a viewport trap using complex event interceptors. The system overrides standard scrolling behavior using a dual-state lock.

The Multi-Layer Interception System:

- **Snapping Scroll Trap:** A global scroll listener locks the viewport at absolute top (`window.scrollY = 0`) while the interactive sequence progress is between 0.0 and 0.999. If the user attempts to scroll the page using scrollbars, the browser viewport instantly snaps back.
- **Mouse Wheel Interception:** The mouse `wheel` event is registered with `{ passive: false }`. Calling `e.preventDefault()` stops the browser's default scroll action. A scroll velocity delta is instead computed: `deltaY * 0.0004`, updating the virtual timeline progress.
- **Mobile Touch Swipe Integration:** Touch starts register the initial touch coordinates on `touchstart`. Subsequent drag actions trigger `touchmove`, calculating a vertical travel delta. This swipe distance is scaled by a fine-tuned touch factor (`0.0012`) and added to the progress timeline, ensuring full mobile touch compatibility.
- **Hardware Keyboard Trapper:** Keydown events are listened to for navigation keystrokes like `ArrowDown`, `ArrowUp`, `PageDown`, `PageUp`, and `Space`. These are blocked via preventDefault, updating the sequence progress by fine-tuned numeric increments (e.g., +/- 0.01 for arrows).
- **Dynamic Speed Dampening:** To ensure highly comfortable reading windows, the system automatically slows down scroll progress velocity by **3.5x** as soon as any text block slides into its final, fully visible position. This dampening is active across three designated reading zones: *0.04 to 0.26*, *0.34 to 0.56*, and *0.64 to 0.86*. When crossing between these zones, the velocity naturally speeds up to produce snappy visual transitions.
- **Unlocking Threshold:** The moment `sequenceProgress` reaches exactly 1.0, the event interceptors release their active preventDefault traps. This allows the page to naturally scroll down to **Section 2: The Operational Edge** and **Section 3: Featured Impact**.

Interception Lifecycle Logic:

```
// Event Interception & Timeline Scrubbing Hook in ScrollyCanvas.tsx const handleWheel = (e: WheelEvent) => { const isAtTop = window.scrollY <= 2; if (!isAtTop) return; if (!isSequenceFinished) { e.preventDefault(); const delta = e.deltaY; const speed = 0.0004; let nextProgress = sequenceProgress.get() + delta * speed; nextProgress = Math.max(0, Math.min(1, nextProgress)); sequenceProgress.set(nextProgress); if (nextProgress === 1 && delta > 0) { setIsSequenceFinished(true); } } else if (isSequenceFinished && e.deltaY < 0) { e.preventDefault(); setIsSequenceFinished(false); // lock scroll and scrub back up... } };
```

4. Synchronized Text Overlay Engine

As the background cinematic frames scroll, a synchronized textual layer displays key titles and subtitles. The text animations must perfectly coordinate with the frame indices to create a high-fidelity 'interactive presentation' effect.

A. Scroll-Triggered Progress Ranges

To map the visual progression of text overlay items precisely, the raw scroll progress (0.0 to 1.0) is split into three continuous active ranges. Within each range, custom Framer Motion hooks smoothly interpolate opacity, x, and y offsets relative to the scroll speed:

Scroll Range	Section State	Visual Layout & Animation
Scroll 0.00 - 0.28	01 // LEADERSHIP	Right-aligned. Slides in dynamically from the right (+120px to 0px) and fades out left.
Scroll 0.28 - 0.57	02 // STRATEGIC DELIVERY	Left-aligned. Slides in dynamically from the left (-120px to 0px) and fades out right.
Scroll 0.57 - 0.95	03 // COGNITIVE OPERATIONS	Centered. Slides up dynamically from the bottom (+100px to 0px) and exits upwards.

B. Continuous Scroll-Driven Interpolation

To maximize physics-based fidelity, the overlays are completely decoupled from active mount/unmount states. Instead, each section is continuously active and bound directly to the global `scrollYProgress` using Framer Motion's `useTransform` hook. When the user scrolls, the elements dynamically translate their coordinates—sliding Section 1 in from the right, Section 2 from the left, and Section 3 up from the bottom. This physical mapping coordinates perfectly with the scrolling gesture, offering an extremely smooth interactive feel.

C. Real-time Progress Tracking Indicators

A technical sub-element is present at the bottom of the viewport: a real-time progress bar. This element maps the raw `scrollYProgress` directly to the width percentage of an active gradient element. The gradient flows across three points (`from-blue-500 via-indigo-500 to-purple-500`), offering a sleek and interactive visual indicator that mirrors the user's progress through the presentation.

5. Core Pillars & Selected Impact Components

Once the user completes the scrollytelling sequence and scrolls down, the interface shifts to a dark, high-contrast, glassmorphic layout showcasing Diwas's operational pillars and real-world metrics.

A. The Pillars Component: Glassmorphism & Radial Hover Glows

The **Pillars** component features a three-column grid outlining key technical competencies. Each card is engineered using customized Tailwind glassmorphic parameters (`glass-card`) with extremely subtle translucent borders. Dynamic hover effects are handled using absolute-positioned radial gradients with large blur radii (`filter: blur(xl)`). These gradients translate on mouseover, causing the cards to glow organically in response to the user's cursor position.

B. The Selected Projects Component: STAR Narrative Timelines

Rather than utilizing standard layouts, the **Projects** component displays impact metrics using a structured **STAR** (Situation, Action, Result) chronological timeline. This is represented as an interactive vertical node system. Visual indicators are connected via a vertical timeline rule (`border-l border-white/10`) and illuminated by styled dot indicators in various accent colors (Blue for Situation, Indigo for Action, Purple for Result). This structure highlights not only the project details but also the measurable impact of each initiative.

C. Secure Contact Endpoints & Footers

The call to action is built around a customized 'Initiate Conversation' button utilizing an active blue drop shadow glow. Contact endpoints are laid out in a clear grid showing system availability, timezone offsets (Kathmandu, Nepal / GMT+5:45), and links to GitHub and LinkedIn. This maintains the clean, tech-oriented look of the portfolio all the way to the footer.

Competency / Project	Target Metric	Measurable Strategic Outcome
AI Workflow Automation	40% faster tasks	Replaced manual bottlenecks with multi-agent AI workflows, achieving 80% automated routines.
Tech Learning Hub	2 Weeks ahead	Delivered construction of learning space for 5,000+ members ahead of schedule.
Digital Upskilling	80% got jobs	Developed a modern tech curriculum, placing 1,200+ trained specialists directly into technical roles.

6. Style Systems & Performance Optimizations

Maintaining high performance while managing large animations and images is crucial. The project implements several performance and optimization techniques to ensure the site runs smoothly.

A. Tailwind CSS v4 & PostCSS Configuration

Tailwind CSS v4 introduces significant improvements in compilation speed and asset optimization. By moving away from JavaScript configurations, v4 relies on a native CSS parser. The PostCSS config uses the new `@tailwindcss/postcss` plugin, which compiles styles directly into light CSS, reducing overall styling overhead and improving initial paint times.

B. Next-Gen Image Optimization

All visual preview banners in the Projects section leverage the Next.js `Image` component. This automatically compresses assets into AVIF/WebP formats, limits widths using responsive layout rules, and handles layout shifts using exact image dimensions. It also leverages lazy-loading on offscreen items, improving the site's Largest Contentful Paint (LCP) score.

C. Zero Bundle-Size Server Components

The main entry point (`src/app/page.tsx`) is a React Server Component. This means the high-level page layout, including imports for individual sections, is processed on the server. Only the interactive components like the ScrollyCanvas and the Framer Motion elements send client-side JavaScript bundles. This approach improves performance, resulting in faster load times and smoother initial rendering.

D. Detailed CSS Declarations (`globals.css`)

The design system utilizes custom CSS declarations. For instance, the glassmorphic cards and glows are defined in `globals.css` using the new Tailwind v4 syntax. Let's look at the implementation:

```
/* Tailwind CSS v4 custom declarations in globals.css */ @import "tailwindcss"; @theme {
--color-background: #121212; --color-card-dark: #0D0D11; } /* Glassmorphic utilities */ .glass-card {
background: rgba(13, 13, 17, 0.7); backdrop-filter: blur(16px) saturate(120%);
-webkit-backdrop-filter: blur(16px) saturate(120%); } /* Background glow ambience */ .glow-bg {
position: absolute; border-radius: 50%; filter: blur(120px); pointer-events: none; z-index: 0; }
```